

高级语言程序设计实验报告

南开大学 软件工程

姓名： 许睿哲

学号： 2412600

班级： 3137

2026年 5月13日

目录

- [一、 作业题目](#)
- [二、 开发软件](#)
- [三、 课题要求](#)
- [四、 主要流程](#)
 - [1. 整体架构与类设计思路](#)
 - [2. 核心功能与算法流程](#)
- [五. 单元测试（测试与验证）](#)
- [六. 收获](#)
- [附录](#)

一、作业题目

- PIXEL ADVENTURE (基于 C++ 与 WebAssembly 的 2D 网页图形化 RPG 游戏)

二、开发软件

- IDE编辑器: Visual Studio Code (VSCode)
- 构建与编译工具: CMake、Emscripten (用于将 C++ 代码编译为 WebAssembly)
- 图形化与前端展示: WebAssembly (Wasm) 结合 HTML5 Canvas / JavaScript 交互

三、课题要求

1. **面向对象设计**: 合理使用 C++ 的类、继承、多态等面向对象特性 (如提取游戏角色的基类, 派生出玩家和怪物类)。
2. **图形化界面**: 突破传统的控制台程序, 利用 WebAssembly 技术将 C++ 逻辑渲染到网页图形化平台中, 实现跨平台的图形化小游戏开发。
3. **创新与创意**: 设计完整的游戏地图 (地牢风格)、角色控制、碰撞检测、攻击逻辑以及动态背景音乐 (BGM), 提升项目的完整度与可玩性。
4. **工程化与版本控制**: 使用 Git 进行版本迭代管理, 体现完整的软件开发流程。

四、主要流程

1. 整体架构与类设计思路

本游戏项目摒弃了传统的深度继承树体系, 采用了现代游戏开发中更灵活的**数据驱动 (Data-Driven)** 与**逻辑分离**架构。整体核心类设计主要分为“数据模型 (Role)”与“逻辑控制器 (Controller)”两大模块:

- **数据模型层 (Model)**:
 - **CharacterRole**: 定义玩家角色的基础数值, 内部包含 Stats (生命值、攻击力)、AttackProfile (攻击范围、目标优先级) 等, 例如:

```
// 创建弓箭手角色: 高攻击力、独特技能 (祝福、滞空箭)
CharacterRole CharacterRole::legendaryLineArcher() {
    CharacterRole role{};
    role.kind_ = RoleKind::LegendaryLineArcher;
    role.displayName_ = "只射直线的传奇弓箭手";
    role.stats_.maxHp = 300;
    role.stats_.attackPower = 50;
    role.normalAttack_.name = "直线发射箭矢";
    role.normalAttack_.rangeTiles = 10;
    role.normalAttack_.autoLock = false;
    role.normalAttack_.hasAreaEffect = false;
    role.smallSkill_.name = "小技能";
    role.smallSkill_.enabled = true;
```

```

role.bigSkill_.name = "大技能";
role.bigSkill_.enabled = true;
return role;

```

- **MonsterRole**：定义怪物的数据模型，额外增加了 **VisionProfile**（视野范围）用于 AI 索敌判定，例如：

```

// 创建哥布林：低级敌人，血量/伤害均衡
MonsterRole MonsterRole::goblin() {
    MonsterRole role{};
    role.displayName_ = "哥布林";
    role.stats_.maxHp = 300;
    role.stats_.attackPower = 20;
    role.normalAttack_.name = "哥布林重击";
    role.normalAttack_.rangeRadius = 1;
    role.normalAttack_.cooldownMs = 1000;
    role.normalAttack_.variants = 2;
    role.vision_.discoverRadius = 2;
    return role;
}

```

- **逻辑控制层（Controller）**：

- **PlayerController**：持有 **CharacterRole** 实例，负责处理玩家的输入请求（如 **requestMove**，**requestAttack**），并维护玩家的帧状态机（是否移动中、是否攻击中、技能冷却回合计数等）。

```

// 更新玩家状态：处理移动/攻击动画、检查动作完成、尝试开始新动作
void PlayerController::update(float nowMs,
    const std::function<bool(std::int32_t, std::int32_t)>& isBlocked,
    const std::function<bool(std::int32_t, std::int32_t)>& hasEnemy,
    bool allowActionStart) {
    if (hurt_ && (nowMs - hurtStartTimeMs_ >= config_.hurtDurationMs))
    {
        ...
    }
    if (dead_) {
        ...
    }
    if (moving_) {
        ...
    }
    if (attacking_) {
        ...
    }
    if (allowActionStart) {

```

```

    ...
}
}

```

- **MonsterController**：持有 **MonsterRole** 实例，负责根据玩家的位置和自身视野执行 AI 逻辑（如随机巡逻、最短路径追击）。

```

// 每当玩家回合推进时执行怪物AI逻辑：技能冷却、探测、巡逻、追击、攻击
void MonsterController::onPlayerTurnAdvanced(
    std::int32_t playerTurn,
    TilePos playerTile,
    float nowMs,
    const std::function<bool(std::int32_t, std::int32_t)>& isBlocked) {
    if (playerTurn <= lastProcessedPlayerTurn_) return;
    lastProcessedPlayerTurn_ = playerTurn;
    if (dead_ || removed_ || move_.active || attacking_) return;
    if (castingSkillId_ != 0) return;
    if (globalSkillCooldown_ > 0) globalSkillCooldown_--;
    for (auto& cd : skillCooldowns_) if (cd > 0) cd--;
    if (role_.displayName() == "哥布林大王" && globalSkillCooldown_ <= 0)
    {
        ...
    }
    if (!discovered_ && inDiscoverRange(playerTile)) {
        discovered_ = true;
    }
    if (!discovered_) {
        randomPatrol(nowMs, isBlocked);
        return;
    }
    faceTo(playerTile);
    if (inAttackRange(playerTile)) {
        startAttack(nowMs);
        return;
    }
    const auto nextStep = nextShortestStepToward(playerTile, isBlocked);
    if (nextStep.has_value()) {
        startMove(nextStep.value(), nowMs);
    }
}

// 使用广度优先搜索找到通往目标的下一步最短路径，考虑障碍物和搜索范围限制
std::optional<TilePos> MonsterController::nextShortestStepToward(
    TilePos target,
    const std::function<bool(std::int32_t, std::int32_t)>& isBlocked)

```

```
const {
    ...
}
```

- **基础类型支持 (Types)**: 统一定义了 Vec2 (世界坐标)、TilePos (瓦片网格坐标) 和 Facing (朝向), 为地图渲染和碰撞检测提供数学基础。

```
struct Vec2 {
    float x = 0.0f;
    float y = 0.0f;
};
struct TilePos {
    std::int32_t x = 0;
    std::int32_t y = 0;
};
enum class Facing {
    Left,
    Right,
    Up,
    Down
};
```

核心类图关系如下:

- PlayerController ——(包含)——> CharacterRole
- MonsterController ——(包含)——> MonsterRole
- 两大 Controller 均依赖 Types.hpp 中的基础几何数据结构。

2. 核心功能与算法流程

本游戏的核心流程在一个按帧更新的 `update(float nowMs)` 函数中推进, 包含以下几个关键算法:

- (1) **坐标转换与移动插值算法** 游戏采用“网格+连续空间”的混合坐标系。逻辑判定在网格 (TilePos) 上进行, 而图形渲染平滑移动则在世界坐标 (Vec2) 上完成。

- 在触发移动时, 起点和终点网格确定, 计算出真实的物理目标点 `targetWorld`。

```
// 处理移动完成: 更新位置, 重置状态, 推进回合
void PlayerController::finishMove(float nowMs,
    const std::function<bool(std::int32_t, std::int32_t)>& isBlocked) {
    tilePos_ = moveTargetTile_;
    worldPos_ = moveTargetWorld_;
    moving_ = false;
    (void)nowMs;
```

```

        (void)isBlocked;
        onTurnAdvanced();
    }

```

- 根据当前时间戳 `nowMs` 与移动起始时间 `moveStartTimeMs_` 的差值，结合配置的移动总时长 `moveDurationMs`，在两点之间进行平滑的线性插值计算，实现像素级的流畅移动表现。

```

// 更新玩家状态：处理移动/攻击动画、检查动作完成、尝试开始新动作
void PlayerController::update(...)
{
    ...
    if (moving_) {
        const float elapsed = nowMs - moveStartTimeMs_;
        const float progress = clamp01(elapsed / config_.moveDurationMs);
        const float eased = easeInOutQuad(progress);
        worldPos_.x = lerp(moveStartWorld_.x, moveTargetWorld_.x, eased);
        worldPos_.y = lerp(moveStartWorld_.y, moveTargetWorld_.y, eased);
        if (progress >= 1.0f) {
            finishMove(nowMs, isBlocked);
        }
    }
    ...
}

```

(2) 怪物 AI 与寻路逻辑 `MonsterController` 中实现了一套轻量级的回合制索敌与追击 AI：

- **视野判定：**通过 `inDiscoverRange` 方法计算怪物与玩家所在的 `TilePos` 之间的网格距离，若小于配置的 `discoverRadius`（默认2格，即5×5范围），则进入发现状态。

```

// 判断玩家是否在怪物的发现范围内（基于曼哈顿距离）
bool MonsterController::inDiscoverRange(TilePos playerTile) const {
    const std::int32_t dx = std::abs(playerTile.x - tilePos_.x);
    const std::int32_t dy = std::abs(playerTile.y - tilePos_.y);
    return dx <= role_.vision().discoverRadius && dy <=
role_.vision().discoverRadius;
}

```

- **最短路径追击：**当玩家进入视野但不在攻击范围（`inAttackRange`）内时，怪物会调用 `nextShortestStepToward` 算法（广度优先）。该算法计算怪物当前位置四邻域网格到玩家位置的距离，结合通过传入的回调函数（`isBlocked`）判定障碍物，选择一个无障碍且距离玩家最近的网格作为下一步移动目标。

(3) 战斗判定与状态机 角色的状态机由一系列布尔标志位管理（如 `moving_`，`attacking_`，`hurt_`，`dead_`）：

- 玩家或怪物发起攻击时，设置 `attacking_ = true` 并记录 `attackStartTimeMs_`。

```
// 更新玩家状态：处理移动/攻击动画、检查动作完成、尝试开始新动作
void PlayerController::update(...) {
    ...
    if (attacking_) {
        const float elapsedAttack = nowMs - attackStartTimeMs_;
        const float attackDuration = std::max(1.0f,
config_.attackDurationMs);
        if (!attackImpactResolved_ && elapsedAttack >= attackDuration *
0.5f) {
            attackImpactResolved_ = true;
        }
        if (elapsedAttack >= config_.attackDurationMs) {
            finishAttack(nowMs, isBlocked);
        }
    }
    if (allowActionStart) {
        tryStartNextAction(nowMs, isBlocked, hasEnemy);
    }
    ...
}
```

- 攻击并非瞬间造成伤害，而是基于时间轴（如 `attackDurationMs` 为 420ms）判定动作播放到特定帧（如“命中帧”）时，利用 `hasEnemy` 等回调检测前方网格是否有敌对实体，从而触发 `applyDamage` 扣除 `currentHp_`，若 HP 归零则将状态切换为 `dead_`。

```
// 处理攻击完成
void PlayerController::finishAttack(float nowMs,
    const std::function<bool(std::int32_t, std::int32_t)>& isBlocked) {
    ...
}
```

(4) 角色职业与 BOSS 技能系统设计 游戏不仅实现了基础的攻击与移动，还通过扩展数据模型实现了多样化的角色职业与怪物技能机制：

- **玩家职业技能设计：**
 - **平平无奇的物理魔法使：**定位为近战角色，基础攻击（近战单体斩击）攻击范围为 1 格且带有自动锁定（`autoLock`）功能。其技能体系（小技能与大技能）围绕近身爆发与特定范围伤害（如大剑挥砍或冲击波）设计，适合新手玩家稳扎稳打。

- **只射直线的传奇弓箭手**：定位为远程输出，基础攻击（直线发射箭矢）覆盖高达 10 格的超远距离，但取消了自动锁定（autoLock = false），要求玩家具备较好的走位与瞄准能力。其技能配置独具特色，包括“祝福”状态加成（移动八向射箭）以及“反弹箭”（附加一次撞墙反弹效果），极大增加了操作上限与策略性。

```
// 弓箭手大技能反弹效果处理代码
// 解析并处理玩家的弓箭祝福技能效果，对所有符合条件的敌人造成伤害
void resolveArcherBlessingVolley(float nowMs) {
    if (!g_player) return;
    if (!g_player->consumeArcherVolleyReady()) return;
    if (g_player->isDead()) return;
    const TilePos origin = g_player->tilePos();
    const std::int32_t range = std::max(1, g_player->role().normalAttack().rangeTiles);
    const bool allowTurn = g_player->isArcherBlessingActive();
    const TilePos dirs[] = {
        {-1, 0}, {1, 0}, {0, -1}, {0, 1},
        {-1, -1}, {1, -1}, {-1, 1}, {1, 1}
    };
    for (const TilePos& dir : dirs) {
        resolveArcherArrowTrace(origin, dir, range, allowTurn, nowMs, origin);
    }
}

// 解析并处理玩家的攻击指令，计算箭矢轨迹并对路径上的敌人造成伤害
void resolveArcherArrowTrace(
    TilePos origin,
    TilePos direction,
    std::int32_t maxRange,
    bool allowOneTurn,
    float nowMs,
    TilePos damageSource
) {
    //跳过无效情况
    ...
    for (std::int32_t travelled = 0; travelled < maxRange; ++travelled)
    {
        TilePos next{current.x + dir.x, current.y + dir.y};
        //如果前方被阻挡，且允许转向一次，则尝试转向（优先斜向，再优先左右，再直接掉头）
        if (isBlocked(next.x, next.y)) {
            if (!allowOneTurn || turned) {
                break;
            }
        }
    }
}
```

```

//如果是斜向，检查两个正交方向的阻挡情况，决定转向方式
if (dir.x != 0 && dir.y != 0) {
    const bool blockX = isBlocked(current.x + dir.x,
current.y);
    const bool blockY = isBlocked(current.x, current.y +
dir.y);
    if (blockX && blockY) {...
    } else if (blockX && !blockY) {...
    } else if (!blockX && blockY) {...
    } else {...}
} else {...}
turned = true;
next = TilePos{current.x + dir.x, current.y + dir.y};
if (isBlocked(next.x, next.y)) {
    break;
}
}
current = next;
//如果当前格子有敌人，造成伤害后停止箭头继续前进
MonsterController* enemy = findAliveEnemyAt(current.x, current.y);
if (enemy) {
    enemy->applyDamage(g_player->currentAttackPower(), nowMs,
damageSource);
    break;
}
}
}

```

• BOSS 敌人（哥布林大王）技能AI机制：

- 与普通哥布林（仅有近战重击与较小视野）不同，哥布林大王作为高级首领，不仅在数值上大幅提升（生命值高达 1500，视野半径扩大至 3 格），还引入了复杂的技能随机状态机。
- 在战斗循环中，BOSS 能够依据内部的随机权重算法或冷却计时器，随机释放 3 种不同机制的特殊技能（点燃地板、模仿玩家技能、召唤小怪）。这种设计打破了常规怪物“只会走向玩家并平A”的刻板模式，要求玩家在对抗 BOSS 时必须时刻观察其起手动作，做出预判与闪避，显著提升了关卡挑战性。

五. 单元测试（测试与验证）

由于本项目为图形化动作RPG游戏，与传统的算法题不同，核心逻辑的验证主要集中在角色状态转移、战斗伤害计算以及边界寻路判定。为了确保底层 Controller 与 Role 机制的正确性，针对核心游戏流程设计了以下功能验证案例：

表 1：核心功能测试案例

模块	输入/前置条件	预期输出	测试目的
移动与碰撞判定	玩家所面朝的前方网格存在障碍物（回调函数 <code>isBlocked</code> 返回 <code>true</code> ），玩家发起移动请求。	<code>PlayerController::requestMove</code> 动作被拦截，玩家的世界坐标（ <code>worldPos</code> ）不发生改变。	验证地图边界与障碍物碰撞判定的正确性。
战斗与伤害计算	物理魔法使（基础攻击力50）靠近普通哥布林（最大血量300）并成功判定攻击命中。	哥布林的当前血量 <code>currentHp_</code> 变为250，且触发 <code>hurt_ = true</code> 受伤状态位。	验证角色属性（ <code>Stats</code> ）读取与扣血逻辑的正确传导。
技能范围限制	弓箭手使用普通直线攻击，敌方目标位于11格之外（弓箭手设定射程 <code>rangeTiles</code> 为10格）。	攻击追踪函数返回失效，敌方目标未受到任何伤害。	验证远程攻击的边界范围限制及空间判定逻辑。
状态机流转与死亡	哥布林大王（BOSS）在承受多次攻击后，其生命值 <code>currentHp_</code> 降至0或以下。	怪物状态机的 <code>dead_</code> 标志位立刻置为 <code>true</code> ，阻断其 AI 寻路，并触发播放死亡动画的倒计时。	验证生命值耗尽时，逻辑状态机的层级切换与阻断是否正常。

测试结果与分析：

在开发与联调过程中，通过控制台日志输出与前端画面的实际表现对比，上述核心逻辑的测试结果均符合预期。游戏内实体的画面表现（如移动插值、攻击前摇、受击硬直）与 C++ 底层的数据状态保持了严格的一致性，未发生内存访问越界（如野指针）或状态死锁异常。

六. 收获

1. 面向对象设计的进阶理解（组合与数据驱动）

在初学面向对象时，往往容易陷入“过度依赖继承”的误区。在独立构思和编写本项目时，我深刻体会到了“数据驱动”与“组合优于继承”在实际工程中的巨大优势。通过将角色的数值属性库（`CharacterRole / MonsterRole`）与行为控制逻辑（`Controller`）解耦，不仅避免了庞大且僵硬的类的继承树，还利用工厂方法极大增强了代码的可扩展性，使得后续快速配置新怪物和新职业变得异常简单。

2. 跨平台编译与 WebAssembly 技术的实践

本次大作业让我彻底突破了传统 C++ 控制台（黑框框）程序的局限。为了实现图形化，我查阅资料并学习了 Emscripten 工具链的使用，成功将 C++ 核心逻辑代码编译为 WebAssembly (Wasm) 字节码。结合 JavaScript 与 HTML5 Canvas，我在网页端实现了流畅的跨平台图形

化渲染。这不仅开拓了我的技术视野，也让我初步掌握了底层语言与前端展示界面的交互原理。

3. 游戏状态机与帧驱动机制的掌握

在实现角色操作与怪物 AI 追击时，我学会了抛弃简单的阻塞式编程，转而使用 `update(float nowMs)` 这种基于时间戳的“帧驱动 (Frame-Driven)”模式。通过设计复杂的状态标志位（如 `moving_`、`attacking_`、`bigSkillCasting_` 等），我深入理解了有限状态机 (Finite State Machine) 的思想。这种控制复杂交互流转的设计模式，对我未来开发任何带有实时交互逻辑的软件系统都有着极为深远的启发。

附：

本项目中利用了 AI 工具辅助进行了代码梳理与排版润色，具体用途如下：

1. Google Gemini 3.1 Pro：用于代码思路和架构的协同梳理
2. GitHub Copilot Pro：用于代码辅助润色和简化，以及算法思路的参考
3. Claude Sunset 4.6：用于代码bug辅助纠错
4. GPT 5.3：用于查找相关资料和指导建议
5. 豆包：用于查询一些简单的基础性知识点